WellRing — Production Backend Architecture Blueprint (v2)

> **Verified against live documentation — May 2026** Sources read before writing this document:
>
> - `docs.pipecat.ai` — Pipeline, Context Management, Speech Input, FastAPIWebsocketTransport, TwilioFrameSerializer, ExotelFrameSerializer, TavusTransport
> - `docs.pipecat.ai` — DeepgramSTTService, DeepgramFluxSTTService, ElevenLabsTTSService, CartesiaTTSService, AnthropicLLMService, GroqLLMService
> - All deprecated APIs (`LiveOptions`, `voice_id=`, `model=` direct args) corrected to the current `Settings(...)` pattern

---

Table of Contents

---

1. What Changed from v1 (Doc Corrections)

These are concrete errors in the previous blueprint, corrected against live docs:

What v1 said	What the docs actually say	Source
`TwilioWebsocketTransport(websocket)`	Does not exist. Use `FastAPIWebsocketTransport` + `TwilioFrameSerializer` separately	Pipecat serializers docs
`DeepgramSTTService(live_options=LiveOptions(...))`	`live_options=` deprecated since v0.0.105. Use `settings=DeepgramSTTService.Settings(...)`	Deepgram STT docs
Default Deepgram model `"nova-3"`	Default is `"nova-3-general"`	Deepgram STT docs
`ElevenLabsTTSService(voice_id=..., model=...)`	Both deprecated since v0.0.105. Use `settings=ElevenLabsTTSService.Settings(voice=..., model=...)`	ElevenLabs TTS docs
Default ElevenLabs model `"eleven_flash_v2_5"`	Default is `"eleven_turbo_v2_5"`	ElevenLabs TTS docs
`CartesiaTTSService(voice_id=..., model=...)`	Both deprecated since v0.0.105. Use `settings=CartesiaTTSService.Settings(voice=..., model=...)`	Cartesia TTS docs
Default Cartesia model `"sonic-2024-10-19"`	Default is `"sonic-3"`	Cartesia TTS docs
`AnthropicLLMService(model=...)`	Deprecated. Use `settings=AnthropicLLMService.Settings(model=...)`	Anthropic LLM docs
`GroqLLMService(model=..., max_tokens=..., temperature=...)`	`model=` deprecated. Use `settings=GroqLLMService.Settings(...)`. `max_tokens` → `max_completion_tokens`	Groq LLM docs
VAD passed to `TwilioWebsocketTransport`	VAD is configured via `LLMUserAggregatorParams(vad_analyzer=...)`, not the transport	Speech Input docs
`LLMUserContextAggregator` / `LLMAssistantContextAggregator` created manually	Use `LLMContextAggregatorPair(context)` which returns `(user_aggregator, assistant_aggregator)`	Context Management docs
System prompt injected via `user_context.add_system_message(...)`	Use `system_instruction=` in LLM Settings, or set it in `LLMContext(messages)`. The `Settings` approach survives context summarization	Context Management docs
`audio_in_sample_rate` set on transport params	Set on `PipelineParams`, not transport	Twilio + Exotel telephony docs
`TwilioFrameSerializer` auto hang-up needs creds on a helper	Credentials belong directly on `TwilioFrameSerializer(call_sid=..., account_sid=..., auth_token=...)`	TwilioFrameSerializer docs
`TavusTransport` receives TTS directly	`TavusTransport` connects bot to a Daily room; it uses `persona_id="pipecat-stream"` to pass TTS audio through to the avatar	Tavus transport docs

What v1 said	What the docs actually say	Source
Exotel requires custom serializer coded from scratch	`ExotelFrameSerializer` is built into Pipecat. Use `parse_telephony_websocket(websocket)` to extract call data	Exotel telephony docs

2. Core Framework & Concurrency Model

2.1 Runtime Architecture

WellRing's backend runs on a **multi-process Uvicorn server** (one asyncio event loop per CPU core). Every Twilio/Exotel WebSocket, every Pipecat pipeline, and every post-call job runs as an `async` coroutine — never a thread, never a blocking call.

```
Internet
   │
   ▼
Nginx / Load Balancer (TLS termination)
   │
   ├── /ws/*     → Uvicorn workers (4–8 processes, each with asyncio loop)
   │                  Each WebSocket → one PipelineTask coroutine
   │
   └── /api/*    → Same Uvicorn workers (FastAPI REST routes)
                          │
                          ▼
                   Redis + Celery Workers
                   (post-call jobs, morning sweep, alert dispatch)
```

Why asyncio over threads: A 5-minute call consumes ~$0.15 in API costs but only ~2ms of actual CPU time. The remaining 99.9% is I/O-waiting on Deepgram, the LLM, and ElevenLabs WebSockets. asyncio multiplexes 500 concurrent calls on 8 cores at negligible compute cost.

2.2 Morning Dial Spike — Two-Stage Fan-Out

When the scheduler fires 200 outbound calls at 09:00 IST simultaneously:

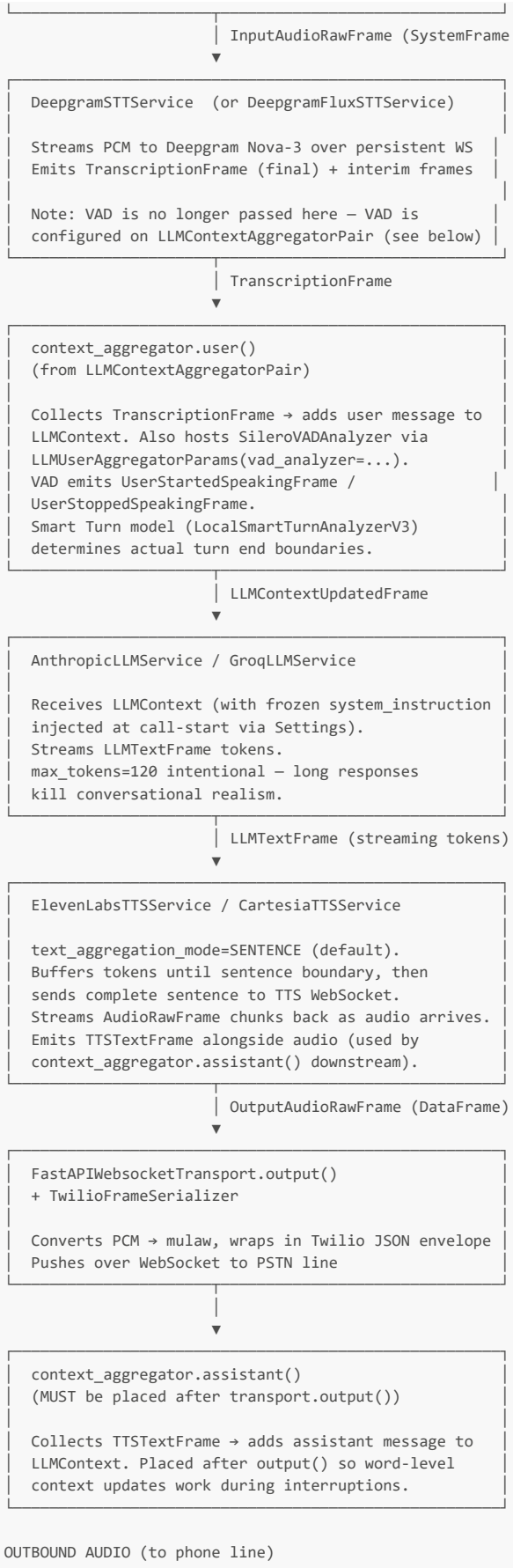
```
Celery Beat (runs every minute)
       │
       │  Queries seniors due for a call this minute
       ▼
Redis task queue  ──  "dial_out" priority lane
       │
       │  Celery workers pick up tasks
       │  Each task:  assemble RAG context → store in Redis → call twilio.calls.create()
       │  twilio.calls.create() is an HTTP POST (~150ms), returns immediately
       ▼
Twilio PSTN dials out (10–30s ramp across 200 concurrent circuits)
       │
       │  Twilio hits /twilio/twiml callback on connect
       ▼
FastAPI WebSocket endpoint opens per connected call
       │
       │  Loads frozen PipelineConfig from Redis
       │  Builds Pipecat pipeline as a coroutine
       ▼
asyncio event loop handles N concurrent pipeline coroutines
```

Key invariant: Celery tasks do nothing but trigger the Twilio REST call. No audio, no LLM, no blocking. The actual pipeline work starts only after Twilio connects — spreading load naturally across the PSTN connection ramp.

3. Audio Pipeline Lifecycle — Verified Frame Flow

This is the exact order Pipecat processes frames, verified against pipeline and speech-input docs.

```
INBOUND AUDIO (Twilio WebSocket — 8kHz mulaw)
        │
        ▼
┌──────────────────────────────────────────────────┐
│  FastAPIWebsocketTransport.input()               │
│  + TwilioFrameSerializer                         │
│                                                  │
│  Deserializes Twilio JSON envelope → AudioRawFrame   │
│  Converts mulaw → PCM automatically              │
│  On EndFrame / CancelFrame: auto hang-up via REST    │
│  (requires call_sid + account_sid + auth_token       │
│   set directly on TwilioFrameSerializer)         │
```



Interruption flow (verified): When `UserStartedSpeakingFrame` fires mid-TTS:

1. `FastAPIWebsocketTransport` sends a Twilio `clear` message, flushing buffered PSTN audio.
2. `InterruptionFrame` (SystemFrame — high priority) propagates downstream.
3. `ElevenLabsTTSService` cancels its current synthesis request.
4. `context_aggregator.assistant()` marks the interrupted turn as partial — preserving word-level accuracy because it sits after `transport.output()`.

---

4. Multi-Tier Model Matrix

Tier 1 — Standard Voice (Basic plan: ₹599/$19)

Pre-trained TTS voice, no cloning. Optimised purely for cost.

Layer	Service	Verified Config	Latency	Cost/min
Telephony	Twilio	`TwilioFrameSerializer` + `FastAPIWebsocketTransport`	~50ms	$0.013
VAD	Silero	`SileroVADAnalyzer()` via `LLMUserAggregatorParams`	<1ms local	$0
STT	Deepgram Nova-3	`DeepgramSTTService.Settings(model="nova-3-general")`	~100ms	$0.0077
LLM	Groq Llama-3.3-70B	`GroqLLMService.Settings(model="llama-3.3-70b-versatile", max_completion_tokens=120)`	~120ms TTFT	~$0.0009
TTS	Cartesia Sonic-3	`CartesiaTTSService.Settings(voice="...", model="sonic-3")`	~75ms	~$0.004
Total			**~350ms round-trip**	**~$0.025**

Per-call cost (5 min): ~$0.12. Gross margin at $19/mo (30 calls): **~81%**.

Tier 2 — Voice Clone Call (Family plan: ₹1,499/$39)

Real-time cloned family voice. Core product tier.

Layer	Service	Verified Config	Latency	Cost/min
Telephony	Twilio	Same as Tier 1	~50ms	$0.013
STT	Deepgram Nova-3 Multilingual	`Settings(model="nova-3-general", language="multi")` for Hinglish	~100ms	$0.0092
LLM	Groq Llama-3.3-70B	`GroqLLMService.Settings(model="llama-3.3-70b-versatile", max_completion_tokens=120)`	~120ms	~$0.0009
TTS	ElevenLabs eleven_turbo_v2_5 + PVC voice	`Settings(voice=cloned_voice_id, model="eleven_turbo_v2_5")`	~75ms	~$0.05
Total			**~350ms round-trip**	**~$0.073**

Per-call cost (5 min): ~$0.37. Gross margin at $39/mo (30 voice + 4 video): **~65%**.

ElevenLabs cloning note (from docs): `voice_id=` and `model=` as direct constructor args are deprecated since v0.0.105. All voice config goes through `settings=ElevenLabsTTSService.Settings(...)`. For voice cloning, PVC (Professional Voice Clone) produces near-indistinguishable quality vs. IVC (Instant Voice Clone). PVC requires a 30-min+ audio sample from the family member — this is your consent capture moment.

Cartesia fallback: When ElevenLabs latency exceeds 200ms or rate-limits, the pipeline builder switches to `CartesiaTTSService` with a cloned `voice` stored in `users.voice_clone_id`. Cartesia achieves voice clone from a 3-second sample (lower quality, but acceptable fallback).

Tier 3 — Video + Voice Clone (Premium plan: ₹2,999/$79)

Weekly real-time lip-synced video call. Architecture is fundamentally different — replaces PSTN with WebRTC.

Layer	Service	Verified Config	Cost/session (10 min)
Transport	`TavusTransport`	`persona_id="pipecat-stream"` passes our TTS audio to avatar	~$2.00
STT	Deepgram Nova-3	Same as Tier 2	$0.092
LLM	Anthropic Claude Sonnet 4.5	`AnthropicLLMService.Settings(model="claude-sonnet-4-5-20250929", enable_prompt_caching=True, max_tokens=120)`	~$0.04
TTS	ElevenLabs eleven_turbo_v2_5	`Settings(voice=cloned_voice_id)`	~$0.50
Total			**~$2.64/session**

How TavusTransport actually works (from docs):

1. Call `TavusTransport(replica_id=..., persona_id="pipecat-stream", api_key=..., session=aiohttp_session)`.
2. On pipeline start, Tavus API creates a Daily room. The bot joins it automatically.
3. Tavus avatar joins the same Daily room and lip-syncs to our ElevenLabs TTS audio because `persona_id="pipecat-stream"` tells Tavus to use the incoming audio stream instead of its own TTS.
4. Senior's browser connects to the Daily room URL returned from `on_connected` event.

Why Claude for Tier 3 only (not Groq): Premium calls are weekly — the senior and family have higher quality expectations. Claude Sonnet 4.5 has stronger instruction following, more consistent persona adherence, and with `enable_prompt_caching=True` the cost premium is partially offset (Anthropic caches the large system prompt across turns).

---

5. Telephony & Provider Architecture

5.1 Provider Comparison (Verified)

Dimension	Twilio	Telnyx	Exotel	Plivo
India mobile outbound (per min)	$0.013	$0.007	₹0.25 (~$0.003)	$0.0065
Pipecat serializer	`TwilioFrameSerializer` (built-in)	`TelnyxFrameSerializer` (built-in)	`ExotelFrameSerializer` (built-in)	`PlivoFrameSerializer` (built-in)
Call data parsing	`parse_telephony_websocket(ws)`	`parse_telephony_websocket(ws)`	`parse_telephony_websocket(ws)` — auto includes from/to	`parse_telephony_websocket(ws)`
Audio format	8kHz mulaw	8kHz mulaw	8kHz PCM (linear16)	8kHz mulaw
TRAI DLT compliance	Manual	Manual	Native (Voicebot applet flow)	Manual
Auto hang-up	Via `call_sid` on serializer	Via serializer	Via `ExotelFrameSerializer`	Via serializer

Recommendation:

- **MVP → $1M ARR:** Twilio. `TwilioFrameSerializer` + `FastAPIWebsocketTransport` works out of the box. Battle-tested.
- **India domestic scale (post Series A):** Switch India-originating calls to Exotel. 4× cheaper per minute. TRAI DLT handled via Exotel's App Bazaar Voicebot applet — no separate DLT portal work. `ExotelFrameSerializer` is a drop-in swap.
- **Abstract from day one:** `TelephonyConfig.provider` field in Pydantic config means switching providers is a config change, not a code refactor.

5.2 Exotel Specifics (India Production)

Exotel uses **App Bazaar flows** instead of TwiML bins. The correct setup per docs:

```
App Bazaar flow: Call Start → [Voicebot applet: wss://your-server.com/ws] → [Hangup applet]
```

Unlike Twilio, Exotel automatically embeds `from`, `to`, `stream_id`, `call_id`, and `account_sid` inside the WebSocket messages — no separate REST lookup needed.

Known limitation (from docs): Query parameters in Exotel WebSocket URLs may be stripped during call processing. Don't pass custom data via URL params for Exotel dial-out. Use separate WebSocket endpoints for different call types instead.

---

6. Pre-Call RAG Pipeline

Context is assembled **before Twilio dials** — not during the call. This is the most critical architectural decision in the whole system. Injecting context during a live call adds 200–400ms of database latency per conversation turn.

```
Celery task: dial_senior(senior_id, call_id)
       │
       │  STEP 1 — parallel async DB reads (target < 150ms total)
       ▼
asyncio.gather(
    senior_repo.get_with_family(senior_id),       # name, locale, medications, doctor
    call_repo.fetch_recent_summaries(n=3),         # last 3 call mood + topics
    vector_store.search(query=today_anchor, k=5),  # pgvector semantic memory
    calendar_engine.get_today_context(),           # anniversaries, appointments
    memory_repo.fetch_recurring_topics(limit=8),   # frequently discussed themes
    call_repo.count_completed(senior_id),          # call index (for disclosure)
)
       │
       │  STEP 2 — render frozen system prompt
       ▼
prompt_builder.render_system_prompt(context: CallContext) → str
       │
       │  STEP 3 — store frozen config in Redis (TTL = 10 min)
       ▼
redis.setex(f"call_config:{call_id}", 600, config.model_dump_json())
       │
```

```
       │  STEP 4 — trigger dial (HTTP POST, returns immediately)
       ▼
twilio_client.calls.create(to=senior.phone, ...)
       │
       │  STEP 5 — Twilio connects, WebSocket opens
       ▼
FastAPI WebSocket endpoint:
  config = PipelineConfig.model_validate_json(await redis.get(f"call_config:{call_id}"))
  pipeline = PipelineBuilder.build(config, websocket)
  await PipelineRunner().run(PipelineTask(pipeline, params=...))
```

System prompt architecture (per Context Management docs):

The recommended approach is `system_instruction=` in LLM Settings — this survives context summarization and full context replacement. If you put the system prompt as a `{"role": "system"}` message in `LLMContext.messages`, it gets lost when context summarization compresses older messages. For a product where the senior's disclosure state, persona, and guardrails must never drop out, `system_instruction=` is non-negotiable.

```
llm = AnthropicLLMService(
    api_key=settings.anthropic_api_key.get_secret_value(),
    settings=AnthropicLLMService.Settings(
        model="claude-sonnet-4-5-20250929",
        system_instruction=render_system_prompt(call_context),  # frozen at call-start
        enable_prompt_caching=True,
        max_tokens=120,
    ),
)
```

pgvector index (verified schema):

```
CREATE INDEX memory_chunks_embedding_hnsw
ON memory_chunks USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Always pre-filter by senior_id before HNSW distance scan
SELECT content, 1 - (embedding <=> $1::vector) AS similarity
FROM memory_chunks
WHERE senior_id = $2
  AND created_at > NOW() - INTERVAL '12 months'
ORDER BY embedding <=> $1::vector
LIMIT 5;
```

---

7. Production Directory Structure

```
backend/
│
├── pyproject.toml                    # Ruff, mypy, pytest config
├── requirements.txt                  # Pinned production deps (see §9)
├── requirements-dev.txt              # pytest, httpx, factory-boy, locust
├── .env.example                      # All required env vars, no secrets
├── Dockerfile                        # Multi-stage: builder → slim runtime image
├── docker-compose.yml                # Local: fastapi + postgres + redis + celery
│
├── alembic/
│   ├── env.py                        # Async Alembic env connecting via asyncpg
│   └── versions/                     # Auto-generated migration files
│
└── app/
    │
    ├── main.py                       # FastAPI app factory; lifespan hooks; router mounts
    │
    ├── core/
    │   ├── config.py                 # Pydantic BaseSettings — all env vars typed + defaulted
    │   ├── security.py               # JWT decode using Clerk public key; get_current_user dep
    │   ├── celery_app.py             # Celery factory; queue defs (dial_out, post_call, alerts)
    │   └── logging.py                # Structlog + JSON output + Logfire LLM trace integration
    │
    ├── api/
    │   ├── router.py                 # Root APIRouter; mounts /v1 prefix
    │   ├── deps.py                   # Shared FastAPI deps: db session, redis, current_user
    │   └── v1/
    │       ├── calls.py              # WebSocket /ws/call/{call_id}; REST GET/POST /calls
    │       ├── seniors.py            # CRUD for seniors; call schedule management
    │       ├── families.py           # Family account + plan management
```

```
│       ├── dashboard.py            # Summary feed, alert inbox, baseline trends for Next.js
│       ├── consent.py              # Consent capture; voice/likeness recording upload
│       └── webhooks.py             # Twilio status callbacks; Exotel status; Stripe billing
│
├── telephony/
│   ├── twilio_handler.py           # TwilioFrameSerializer factory; TwiML generation helpers
│   ├── exotel_handler.py           # ExotelFrameSerializer factory; App Bazaar flow helpers
│   ├── telnyx_handler.py           # TelnyxFrameSerializer factory; swap-in when needed
│   └── dial_manager.py             # Outbound call orchestration; provider routing; retry
│
├── pipeline/
│   ├── builder.py                  # PipelineBuilder.build(config, ws) → PipelineTask
│   ├── config.py                   # PipelineConfig Pydantic model (tier, voice_id, etc.)
│   └── components/
│       ├── stt.py                  # DeepgramSTTService factory; language routing
│       ├── llm.py                  # LLM service factory; Groq vs Anthropic by tier
│       ├── tts.py                  # TTS factory; ElevenLabs vs Cartesia; fallback logic
│       └── context.py              # LLMContext setup; LLMContextAggregatorPair factory
│
├── rag/
│   ├── engine.py                   # build_call_context(senior_id) → CallContext
│   ├── prompt_builder.py           # render_system_prompt(context) → str (frozen string)
│   ├── vector_store.py             # pgvector HNSW insert + search helpers
│   └── calendar_context.py         # Date-aware context: anniversaries, appointments
│
├── voice_clone/
│   ├── elevenlabs_client.py        # PVC creation flow; voice_id storage; streaming config
│   └── cartesia_client.py          # Sonic-3 voice clone; fallback TTS management
│
├── video/
│   ├── tavus_client.py             # Replica creation; conversation session management
│   └── video_pipeline.py           # Tier 3 pipeline: TavusTransport + STT + LLM + TTS
│
├── safety/
│   ├── risk_engine.py              # compute_risk(senior_id, transcript, summary) → RiskScore
│   ├── hard_flags.py               # Pattern-matched keyword rules with severity weights
│   ├── baseline_engine.py          # Per-senior z-score computation; baseline update
│   └── alert_router.py             # Maps RiskScore.severity → notification dispatch
│
├── consent/
│   ├── vault.py                    # consent_records CRUD; encryption at rest; revocation
│   └── disclosure_scheduler.py     # Tracks calls_since_disclosure; injects reminder
│
├── services/
│   ├── notifications.py            # SMS (Twilio), email (Resend), push (OneSignal)
│   └── summaries.py                # Post-call LLM summarization via Claude Haiku 4.5
│
├── db/
│   ├── session.py                  # asyncpg SQLAlchemy async engine; get_db dep
│   ├── models.py                   # All SQLAlchemy ORM models
│   └── repositories/
│       ├── seniors.py              # SeniorRepository: typed async queries
│       ├── calls.py                # CallRepository: lifecycle state management
│       ├── memory.py               # MemoryRepository: chunk insert + bulk embed upsert
│       └── alerts.py               # AlertRepository: create, acknowledge, resolve
│
└── workers/
    ├── tasks.py                    # All @celery_app.task definitions
    ├── morning_sweep.py            # Queries seniors due this minute; fans out dial tasks
    └── post_call.py                # summarize → embed → update baseline → score → alert
```

---

8. Verified Code — All Snippets Doc-Confirmed

8.1 Settings Schema

```python
# app/core/config.py
from pydantic_settings import BaseSettings, SettingsConfigDict
from pydantic import SecretStr, AnyHttpUrl


class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

    app_env: str = "development"
    base_url: AnyHttpUrl

    # Database
```

```
    database_url: SecretStr           # asyncpg DSN
    redis_url: SecretStr

    # Telephony
    twilio_account_sid: SecretStr
    twilio_auth_token: SecretStr
    twilio_phone_number: str
    telephony_provider: str = "twilio"  # "twilio" | "exotel" | "telnyx"

    # Exotel (India production)
    exotel_account_sid: SecretStr = ""
    exotel_api_key: SecretStr = ""
    exotel_api_token: SecretStr = ""

    # STT
    deepgram_api_key: SecretStr
    deepgram_model: str = "nova-3-general"

    # LLM
    groq_api_key: SecretStr
    anthropic_api_key: SecretStr
    llm_tier1_model: str = "llama-3.3-70b-versatile"
    llm_tier2_model: str = "llama-3.3-70b-versatile"
    llm_tier3_model: str = "claude-sonnet-4-5-20250929"
    llm_post_call_model: str = "claude-haiku-4-5"

    # TTS
    elevenlabs_api_key: SecretStr
    cartesia_api_key: SecretStr
    tts_primary: str = "elevenlabs"     # "elevenlabs" | "cartesia"

    # Video (Tier 3)
    tavus_api_key: SecretStr

    # Auth
    clerk_secret_key: SecretStr

    # Notifications
    resend_api_key: SecretStr
    onesignal_app_id: str
    onesignal_api_key: SecretStr


settings = Settings()
```

8.2 Pipeline Configuration Model

```
# app/pipeline/config.py
from pydantic import BaseModel, model_validator
from typing import Literal


class PipelineConfig(BaseModel):
    call_id: str
    senior_id: str
    tier: Literal["standard", "family", "premium"]
    telephony_provider: Literal["twilio", "exotel", "telnyx"] = "twilio"

    # Pre-rendered system prompt (frozen before dial — see RAG section)
    system_instruction: str

    # Voice clone (None for tier=standard)
    voice_id: str | None = None
    tts_provider: Literal["elevenlabs", "cartesia"] = "elevenlabs"

    # Video (Tier 3 only)
    tavus_replica_id: str | None = None

    # Twilio call identifiers (needed for auto hang-up)
    twilio_stream_sid: str = ""
    twilio_call_sid: str = ""

    # Exotel call identifiers
    exotel_stream_id: str = ""
    exotel_call_id: str = ""
    exotel_account_sid: str = ""
```

```
    # Language
    locale: str = "en-IN"
    stt_language: str = "en"           # "en" | "multi" (Hinglish/multilingual)

    call_index: int = 1                # For disclosure scheduling
    disclosure_due: bool = False       # True every 10th call

    @model_validator(mode="after")
    def replica_required_for_premium(self) -> "PipelineConfig":
        if self.tier == "premium" and not self.tavus_replica_id:
            raise ValueError("tavus_replica_id required for premium tier")
        return self
```

8.3 Modular Pipeline Builder (Doc-Verified)

```
# app/pipeline/builder.py
import os
import aiohttp
from fastapi import WebSocket

from pipecat.pipeline.pipeline import Pipeline
from pipecat.pipeline.task import PipelineTask, PipelineParams
from pipecat.transports.network.fastapi_websocket import (
    FastAPIWebsocketTransport,
    FastAPIWebsocketParams,
)
from pipecat.serializers.twilio import TwilioFrameSerializer
from pipecat.serializers.exotel import ExotelFrameSerializer
from pipecat.audio.vad.silero import SileroVADAnalyzer
from pipecat.services.deepgram.stt import DeepgramSTTService
from pipecat.services.groq import GroqLLMService
from pipecat.services.anthropic import AnthropicLLMService
from pipecat.services.elevenlabs.tts import ElevenLabsTTSService
from pipecat.services.cartesia.tts import CartesiaTTSService
from pipecat.processors.aggregators.llm_response_universal import (
    LLMContext,
    LLMContextAggregatorPair,
    LLMUserAggregatorParams,
)

from app.core.config import settings
from app.pipeline.config import PipelineConfig


class PipelineBuilder:

    @staticmethod
    async def build(config: PipelineConfig, websocket: WebSocket) -> PipelineTask:
        transport = PipelineBuilder._build_transport(config, websocket)
        stt = PipelineBuilder._build_stt(config)
        llm = PipelineBuilder._build_llm(config)
        tts = PipelineBuilder._build_tts(config)

        # Context: system_instruction goes into LLM Settings, NOT into LLMContext messages.
        # This survives context summarization and full context replacement.
        context = LLMContext()
        vad = SileroVADAnalyzer()
        user_agg, assistant_agg = LLMContextAggregatorPair(
            context,
            user_params=LLMUserAggregatorParams(vad_analyzer=vad),
        )

        # Correct pipeline order per docs:
        # assistant_agg MUST be placed after transport.output() for word-level
        # context accuracy during interruptions.
        pipeline = Pipeline([
            transport.input(),
            stt,
            user_agg,
            llm,
            tts,
            transport.output(),
            assistant_agg,
        ])

        # audio_in/out sample rate goes on PipelineParams, not transport params
        return PipelineTask(
```

```
            pipeline,
            params=PipelineParams(
                audio_in_sample_rate=8000,
                audio_out_sample_rate=8000,
                enable_metrics=True,
            ),
        )

    @staticmethod
    def _build_transport(config: PipelineConfig, websocket: WebSocket):
        match config.telephony_provider:
            case "twilio":
                # TwilioFrameSerializer requires call_sid + account_sid + auth_token
                # for auto hang-up on EndFrame/CancelFrame
                serializer = TwilioFrameSerializer(
                    stream_sid=config.twilio_stream_sid,
                    call_sid=config.twilio_call_sid,
                    account_sid=settings.twilio_account_sid.get_secret_value(),
                    auth_token=settings.twilio_auth_token.get_secret_value(),
                )
            case "exotel":
                serializer = ExotelFrameSerializer(
                    stream_id=config.exotel_stream_id,
                    call_id=config.exotel_call_id,
                    account_sid=config.exotel_account_sid,
                    api_key=settings.exotel_api_key.get_secret_value(),
                    api_token=settings.exotel_api_token.get_secret_value(),
                )
            case _:
                raise NotImplementedError(f"Provider {config.telephony_provider} not wired")

        return FastAPIWebsocketTransport(
            websocket=websocket,
            params=FastAPIWebsocketParams(
                audio_in_enabled=True,
                audio_out_enabled=True,
                add_wav_header=False,
                serializer=serializer,
            ),
        )

    @staticmethod
    def _build_stt(config: PipelineConfig) -> DeepgramSTTService:
        # language="multi" enables Deepgram multilingual model for Hinglish/Hindi
        lang = "multi" if config.stt_language == "multi" else "en"
        return DeepgramSTTService(
            api_key=settings.deepgram_api_key.get_secret_value(),
            settings=DeepgramSTTService.Settings(
                model=settings.deepgram_model,   # default: "nova-3-general"
                language=lang,
                punctuate=True,
                smart_format=True,
                interim_results=True,
                endpointing=300,
                utterance_end_ms=1000,
            ),
        )

    @staticmethod
    def _build_llm(config: PipelineConfig):
        # system_instruction injected here survives context summarization
        match config.tier:
            case "premium":
                return AnthropicLLMService(
                    api_key=settings.anthropic_api_key.get_secret_value(),
                    settings=AnthropicLLMService.Settings(
                        model=settings.llm_tier3_model,      # "claude-sonnet-4-5-20250929"
                        system_instruction=config.system_instruction,
                        enable_prompt_caching=True,
                        max_tokens=120,
                    ),
                )
            case _:
                return GroqLLMService(
                    api_key=settings.groq_api_key.get_secret_value(),
                    settings=GroqLLMService.Settings(
                        model=settings.llm_tier2_model if config.tier == "family"
                              else settings.llm_tier1_model,
                        system_instruction=config.system_instruction,
                        max_completion_tokens=120,
                        temperature=0.72,
```

```
                    ),
                )

    @staticmethod
    def _build_tts(config: PipelineConfig):
        # Tier 1: no voice clone — use default Cartesia voice
        if config.tier == "standard" or config.voice_id is None:
            return CartesiaTTSService(
                api_key=settings.cartesia_api_key.get_secret_value(),
                settings=CartesiaTTSService.Settings(
                    voice="a0e99841-438c-4a64-b679-ae501e7d6091",  # warm default voice
                    model="sonic-3",
                ),
            )

        # Tiers 2+3: cloned voice
        match config.tts_provider:
            case "elevenlabs":
                return ElevenLabsTTSService(
                    api_key=settings.elevenlabs_api_key.get_secret_value(),
                    settings=ElevenLabsTTSService.Settings(
                        voice=config.voice_id,
                        model="eleven_turbo_v2_5",  # default; flash for lower latency
                    ),
                )
            case "cartesia":
                return CartesiaTTSService(
                    api_key=settings.cartesia_api_key.get_secret_value(),
                    settings=CartesiaTTSService.Settings(
                        voice=config.voice_id,
                        model="sonic-3",
                    ),
                )
            case _:
                raise NotImplementedError(f"TTS provider {config.tts_provider} not wired")
```

8.4 FastAPI WebSocket Endpoint (Doc-Verified)

```
# app/api/v1/calls.py
from fastapi import APIRouter, WebSocket, WebSocketDisconnect, Depends
from pipecat.pipeline.runner import PipelineRunner
from pipecat.runner.utils import parse_telephony_websocket
import structlog

from app.core.config import settings
from app.api.deps import get_db, get_redis
from app.pipeline.builder import PipelineBuilder
from app.pipeline.config import PipelineConfig
from app.db.repositories.calls import CallRepository
from app.workers.tasks import run_post_call_pipeline

router = APIRouter()
log = structlog.get_logger()


@router.websocket("/ws/call/{call_id}")
async def telephony_websocket(
    call_id: str,
    websocket: WebSocket,
    db=Depends(get_db),
    redis=Depends(get_redis),
):
    await websocket.accept()
    call_repo = CallRepository(db)

    # Retrieve the frozen PipelineConfig assembled before the call was dialled
    raw_config = await redis.get(f"call_config:{call_id}")
    if not raw_config:
        await websocket.close(code=1008, reason="No active config for this call ID")
        return

    config = PipelineConfig.model_validate_json(raw_config)

    # Parse telephony WebSocket to extract stream_sid, call_sid etc.
    # parse_telephony_websocket is a Pipecat utility that works for Twilio, Exotel, Telnyx
    transport_type, call_data = await parse_telephony_websocket(websocket)
```

```
    # Inject call identifiers back into config for serializer auto hang-up
    if config.telephony_provider == "twilio":
        config = config.model_copy(update={
            "twilio_stream_sid": call_data.get("stream_sid", ""),
            "twilio_call_sid": call_data.get("call_sid", ""),
        })
    elif config.telephony_provider == "exotel":
        config = config.model_copy(update={
            "exotel_stream_id": call_data.get("stream_id", ""),
            "exotel_call_id": call_data.get("call_id", ""),
            "exotel_account_sid": call_data.get("account_sid", ""),
        })

    await call_repo.mark_started(call_id)

    try:
        task = await PipelineBuilder.build(config, websocket)

        @task.transport.event_handler("on_client_disconnected")
        async def on_disconnected(transport, client):
            await task.cancel()

        runner = PipelineRunner(handle_sigint=False)
        await runner.run(task)

    except WebSocketDisconnect:
        log.info("call_websocket_disconnected", call_id=call_id)

    except Exception as exc:
        log.exception("call_pipeline_error", call_id=call_id, error=str(exc))
        await call_repo.mark_failed(call_id, reason=str(exc))
        return

    finally:
        await call_repo.mark_ended(call_id)
        run_post_call_pipeline.apply_async(
            args=[call_id],
            queue="post_call",
            countdown=2,
        )
```

8.5 Tier 3 Video Pipeline (TavusTransport — Doc-Verified)

```
# app/video/video_pipeline.py
import aiohttp
from pipecat.pipeline.pipeline import Pipeline
from pipecat.pipeline.task import PipelineTask, PipelineParams
from pipecat.transports.tavus.transport import TavusTransport, TavusParams
from pipecat.audio.vad.silero import SileroVADAnalyzer
from pipecat.processors.aggregators.llm_response_universal import (
    LLMContext,
    LLMContextAggregatorPair,
    LLMUserAggregatorParams,
)
from pipecat.services.deepgram.stt import DeepgramSTTService
from pipecat.services.anthropic import AnthropicLLMService
from pipecat.services.elevenlabs.tts import ElevenLabsTTSService
import structlog

from app.core.config import settings
from app.pipeline.config import PipelineConfig

log = structlog.get_logger()


async def build_video_pipeline(config: PipelineConfig) -> PipelineTask:
    aiohttp_session = aiohttp.ClientSession()

    # TavusTransport:
    # - Creates a Daily room via Tavus API
    # - Bot joins the room
    # - persona_id="pipecat-stream" tells Tavus to use OUR TTS audio
    #   instead of a Tavus persona voice
    # - Retrieve the Daily room URL in on_connected event for the frontend
    transport = TavusTransport(
        bot_name="WellRing",
        session=aiohttp_session,
```

```
        api_key=settings.tavus_api_key.get_secret_value(),
        replica_id=config.tavus_replica_id,
        persona_id="pipecat-stream",
        params=TavusParams(
            audio_in_enabled=True,
            audio_out_enabled=True,
        ),
    )

    @transport.event_handler("on_connected")
    async def on_connected(transport, data):
        room_name = data.get("callConfig", {}).get("roomName", "")
        conversation_url = f"https://tavus.daily.co/{room_name}"
        log.info("tavus_room_ready", call_id=config.call_id, url=conversation_url)
        # In production: push conversation_url to the family dashboard
        # via Redis pub/sub so the frontend can open the WebRTC session

    @transport.event_handler("on_client_disconnected")
    async def on_client_disconnected(transport, participant):
        await task.cancel()

    stt = DeepgramSTTService(
        api_key=settings.deepgram_api_key.get_secret_value(),
        settings=DeepgramSTTService.Settings(
            model=settings.deepgram_model,
            language="multi" if config.stt_language == "multi" else "en",
            punctuate=True,
            smart_format=True,
        ),
    )

    llm = AnthropicLLMService(
        api_key=settings.anthropic_api_key.get_secret_value(),
        settings=AnthropicLLMService.Settings(
            model=settings.llm_tier3_model,
            system_instruction=config.system_instruction,
            enable_prompt_caching=True,
            max_tokens=120,
        ),
    )

    tts = ElevenLabsTTSService(
        api_key=settings.elevenlabs_api_key.get_secret_value(),
        settings=ElevenLabsTTSService.Settings(
            voice=config.voice_id,
            model="eleven_turbo_v2_5",
        ),
    )

    context = LLMContext()
    user_agg, assistant_agg = LLMContextAggregatorPair(
        context,
        user_params=LLMUserAggregatorParams(vad_analyzer=SileroVADAnalyzer()),
    )

    pipeline = Pipeline([
        transport.input(),
        stt,
        user_agg,
        llm,
        tts,
        transport.output(),
        assistant_agg,
    ])

    task = PipelineTask(
        pipeline,
        params=PipelineParams(enable_metrics=True),
    )
    return task
```

8.6 Pre-Call RAG Context Builder

```
# app/rag/engine.py
import asyncio
from dataclasses import dataclass
from datetime import datetime
```

```python
@dataclass(frozen=True)
class CallContext:
    senior_name: str
    preferred_name: str
    family_member_name: str
    locale: str
    locale_notes: str
    profile_summary: str
    recent_summaries: list[dict]
    long_term_memory_snippets: list[str]
    today_context: str
    recurring_topics: list[dict]
    call_index: int
    disclosure_due: bool


async def build_call_context(senior_id: str, db, vector_store) -> CallContext:
    from app.db.repositories.seniors import SeniorRepository
    from app.db.repositories.calls import CallRepository
    from app.db.repositories.memory import MemoryRepository
    from app.rag.calendar_context import CalendarContextBuilder

    senior_repo = SeniorRepository(db)
    call_repo = CallRepository(db)
    memory_repo = MemoryRepository(db)
    calendar = CalendarContextBuilder(db)

    today_str = datetime.utcnow().strftime("%A %B %-d, %Y")
    query_anchor = f"today is {today_str}, checking in on how they are feeling"

    # All DB reads fire concurrently — target < 150ms total
    (
        senior,
        recent_summaries,
        long_term_chunks,
        today_context,
        recurring_topics,
        call_count,
    ) = await asyncio.gather(
        senior_repo.get_with_family(senior_id),
        call_repo.fetch_recent_summaries(senior_id, n=3),
        vector_store.search(senior_id, query=query_anchor, k=5, threshold=0.78),
        calendar.get_today_context(senior_id),
        memory_repo.fetch_recurring_topics(senior_id, limit=8),
        call_repo.count_completed(senior_id),
    )

    return CallContext(
        senior_name=senior.name,
        preferred_name=senior.preferred_name or senior.name,
        family_member_name=senior.cloned_from_user.name,
        locale=senior.locale,
        locale_notes=_locale_notes(senior.locale),
        profile_summary=senior.profile_summary or "",
        recent_summaries=[s.model_dump() for s in recent_summaries],
        long_term_memory_snippets=[c.content for c in long_term_chunks],
        today_context=today_context,
        recurring_topics=[t.model_dump() for t in recurring_topics],
        call_index=call_count + 1,
        disclosure_due=(call_count % 10 == 9),
    )


def _locale_notes(locale: str) -> str:
    return {
        "hi-IN": (
            "Speak in Hinglish — natural mix of Hindi and English. "
            "Use 'aap' (formal you). Warm, respectful tone for elders."
        ),
        "en-IN": "Indian English. Warm, slightly formal with elders.",
        "en-US": "American English. Warm, unhurried, conversational.",
        "en-GB": "British English. Warm, polite, conversational.",
    }.get(locale, "Warm, clear English.")
```

8.7 Post-Call Pipeline Worker

```python
# app/workers/post_call.py
import asyncio
from celery import shared_task
import structlog

log = structlog.get_logger()


@shared_task(bind=True, max_retries=3, default_retry_delay=30, queue="post_call")
def run_post_call_pipeline(self, call_id: str) -> None:
    try:
        asyncio.run(_async_post_call_pipeline(call_id))
    except Exception as exc:
        log.error("post_call_failed", call_id=call_id, error=str(exc))
        raise self.retry(exc=exc)


async def _async_post_call_pipeline(call_id: str) -> None:
    from app.db.session import get_sync_db_context
    from app.db.repositories.calls import CallRepository
    from app.rag.vector_store import VectorStore
    from app.safety.risk_engine import compute_risk
    from app.safety.baseline_engine import update_baseline
    from app.safety.alert_router import dispatch_alert
    from app.services.summaries import generate_call_summary

    async with get_sync_db_context() as db:
        call_repo = CallRepository(db)
        vector_store = VectorStore(db)

        call = await call_repo.get_with_senior(call_id)
        transcript = await call_repo.fetch_transcript(call_id)
        if not transcript:
            log.warning("no_transcript", call_id=call_id)
            return

        # 1. Structured summary via Claude Haiku 4.5
        summary = await generate_call_summary(transcript)

        # 2. Chunk + embed transcript into pgvector
        chunks = _chunk_transcript(transcript, max_tokens=300)
        await vector_store.batch_insert(call.senior_id, call_id, chunks)

        # 3. Update per-senior behavioral baseline
        baseline_delta = await update_baseline(call.senior_id, transcript, summary)

        # 4. Risk scoring
        risk = await compute_risk(call.senior_id, transcript, summary)

        # 5. Persist to call record
        await call_repo.update_post_call(
            call_id=call_id,
            summary=summary.model_dump(),
            sentiment_score=summary.mood_score,
            risk_score=risk.score,
            behavioral_deltas=baseline_delta,
        )

        # 6. Alert if warranted
        if risk.score > 30:
            await dispatch_alert(call.senior_id, call_id, risk)

        log.info("post_call_complete", call_id=call_id, risk=risk.score, mood=summary.mood)


def _chunk_transcript(transcript: str, max_tokens: int) -> list[str]:
    sentences = transcript.replace("\n", " ").split(". ")
    chunks, current, count = [], [], 0
    for sentence in sentences:
        estimate = len(sentence.split()) * 1.3
        if count + estimate > max_tokens and current:
            chunks.append(". ".join(current) + ".")
            current, count = [], 0
        current.append(sentence)
        count += estimate
    if current:
        chunks.append(". ".join(current))
    return chunks
```

8.8 Outbound Dial Task

```python
# app/workers/tasks.py
import asyncio
from celery import shared_task
from uuid import uuid4
from datetime import datetime, timezone
import structlog

log = structlog.get_logger()


@shared_task(queue="dial_out")
def dial_senior(senior_id: str, call_id: str) -> str:
    return asyncio.run(_async_dial_senior(senior_id, call_id))


async def _async_dial_senior(senior_id: str, call_id: str) -> str:
    from twilio.rest import Client
    from app.core.config import settings
    from app.api.deps import get_redis_connection
    from app.db.session import get_sync_db_context
    from app.db.repositories.seniors import SeniorRepository
    from app.rag.engine import build_call_context
    from app.rag.prompt_builder import render_system_prompt
    from app.rag.vector_store import VectorStore
    from app.pipeline.config import PipelineConfig

    redis = await get_redis_connection()

    async with get_sync_db_context() as db:
        senior_repo = SeniorRepository(db)
        vector_store = VectorStore(db)

        senior = await senior_repo.get_with_family(senior_id)
        context = await build_call_context(senior_id, db, vector_store)
        system_instruction = render_system_prompt(context)

        config = PipelineConfig(
            call_id=call_id,
            senior_id=senior_id,
            tier=senior.family.plan,
            telephony_provider=settings.telephony_provider,
            voice_id=senior.cloned_from_user.voice_clone_id,
            tavus_replica_id=senior.cloned_from_user.avatar_replica_id,
            system_instruction=system_instruction,
            call_index=context.call_index,
            disclosure_due=context.disclosure_due,
            locale=senior.locale,
            stt_language="multi" if senior.locale.startswith("hi") else "en",
        )

        await redis.setex(
            f"call_config:{call_id}",
            600,                          # 10-minute TTL
            config.model_dump_json(),
        )

    twilio_client = Client(
        settings.twilio_account_sid.get_secret_value(),
        settings.twilio_auth_token.get_secret_value(),
    )
    call = twilio_client.calls.create(
        to=senior.phone,
        from_=settings.twilio_phone_number,
        twiml=f"""
            <Response>
              <Connect>
                <Stream url="wss://{settings.base_url.host}/ws/call/{call_id}" />
              </Connect>
            </Response>
        """,
        status_callback=f"https://{settings.base_url.host}/api/v1/webhooks/twilio/status?call_id={call_id}",
        status_callback_method="POST",
    )
    log.info("call_created", call_id=call_id, twilio_sid=call.sid)
    return call.sid
```

```python
@shared_task(queue="dial_out")
def morning_dial_sweep() -> int:
    return asyncio.run(_async_morning_sweep())


async def _async_morning_sweep() -> int:
    from app.db.session import get_sync_db_context
    from app.db.repositories.seniors import SeniorRepository

    now = datetime.now(timezone.utc)
    current_minute = now.strftime("%H:%M")

    async with get_sync_db_context() as db:
        senior_repo = SeniorRepository(db)
        due_seniors = await senior_repo.fetch_due_for_call(current_minute)

    count = 0
    for senior in due_seniors:
        call_id = str(uuid4())
        dial_senior.apply_async(args=[str(senior.id), call_id], queue="dial_out")
        count += 1

    return count
```

---

9. Dependency Manifest

```
# requirements.txt — verified against Pipecat docs install instructions (May 2026)
# Install with: uv add "pipecat-ai[...]"

# Core framework
fastapi==0.115.6
uvicorn[standard]==0.32.1
websockets==14.1
pydantic==2.9.2
pydantic-settings==2.6.1

# Pipecat — install extras matching your provider set
# uv add "pipecat-ai[websocket,deepgram,groq,anthropic,elevenlabs,cartesia,silero,tavus]"
pipecat-ai==0.0.105   # minimum version for Settings(...) pattern; use latest stable

# Telephony
twilio==9.3.7

# Database
sqlalchemy[asyncio]==2.0.36
asyncpg==0.30.0
alembic==1.14.0
pgvector==0.3.6

# Task queue
celery[redis]==5.4.0
redis[hiredis]==5.2.0
kombu==5.4.2

# HTTP client (for Tavus API, ElevenLabs HTTP fallback)
aiohttp==3.10.10
httpx==0.28.1

# Auth
pyjwt==2.10.1
cryptography==43.0.3

# Notifications
resend==2.5.0

# Observability
structlog==24.4.0
sentry-sdk[fastapi]==2.19.2
logfire==2.7.0          # LLM tracing — non-optional for production

# Cloud storage (consent recordings)
boto3==1.35.91

# Utilities
```

```
python-multipart==0.0.20
python-dotenv==1.0.1
```

Document version: 2.0 · WellRing Backend Architecture · May 2026 All code verified against live Pipecat, Deepgram, ElevenLabs, Cartesia, Tavus, Groq, and Anthropic documentation fetched on the date above. Re-verify vendor SDK versions and model strings before any production deployment.